

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №2
По дисциплине «Интеллектуальный анализ данных»
Тема: «Автоэнкодеры»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Шоева Е.Т.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: научиться применять автоэнкодеры для осуществления визуализации данных и их анализа

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент с использованием нейросетевой модели автоэнкодера (с двумя и тремя нейронами в среднем слое);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки matplotlib, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Реализовать метод t-SNE для визуализации данных (использовать также 2 и 3 компонента), построить соответствующую визуализацию;
4. Применить к данным метод PCA (2 и 3 компонента), реализованный в ЛР №1, сделать выводы;
5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Вариант:

№	Выборка	Класс
11	Wisconsin Diagnostic Breast Cancer (WDBC)	2-й признак

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

data = load_breast_cancer()
X = data.data
y = data.target

scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
colors = {0: 'r', 1: 'g'}
```

```
labels_map = {0: 'malignant', 1: 'benign'}
```

```
def plot_2d(X2, title, filename):
```

```
    plt.figure(figsize=(7, 6))
```

```
    for c in np.unique(y):
```

```
        mask = y == c
```

```
        plt.scatter(X2[mask, 0], X2[mask, 1], c=colors[c], label=labels_map[c], alpha=0.7)
```

```
    plt.xlabel('Component 1')
```

```
    plt.ylabel('Component 2')
```

```
    plt.title(title)
```

```
    plt.legend()
```

```
    plt.tight_layout()
```

```
    plt.savefig(filename)
```

```
    plt.show()
```

```
def plot_3d(X3, title, filename):
```

```
    fig = plt.figure(figsize=(8, 7))
```

```
    ax = fig.add_subplot(111, projection='3d')
```

```
    for c in np.unique(y):
```

```
        mask = y == c
```

```
        ax.scatter(X3[mask, 0], X3[mask, 1], X3[mask, 2], c=colors[c], label=labels_map[c],  
alpha=0.7)
```

```
    ax.set_xlabel('Component 1')
```

```
    ax.set_ylabel('Component 2')
```

```
    ax.set_zlabel('Component 3')
```

```
    ax.set_title(title)
```

```
    ax.legend()
```

```
    plt.tight_layout()
```

```
    plt.savefig(filename)
```

```
    plt.show()
```

```
pca2 = PCA(n_components=2)
```

```
X_pca2 = pca2.fit_transform(X_scaled)
plot_2d(X_pca2, 'PCA (2 components)', 'pca_2d.png')
```

```
pca3 = PCA(n_components=3)
X_pca3 = pca3.fit_transform(X_scaled)
plot_3d(X_pca3, 'PCA (3 components)', 'pca_3d.png')
```

```
print("PCA explained variance ratio (2 comp):", pca2.explained_variance_ratio_.sum())
print("PCA explained variance ratio (3 comp):", pca3.explained_variance_ratio_.sum())
```

```
class Autoencoder(nn.Module):
    def __init__(self, input_dim, latent_dim):
        super(Autoencoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 16),
            nn.ReLU(),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Linear(8, latent_dim)
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 8),
            nn.ReLU(),
            nn.Linear(8, 16),
            nn.ReLU(),
            nn.Linear(16, input_dim)
        )

    def forward(self, x):
        z = self.encoder(x)
        out = self.decoder(z)
        return out, z
```

```
def train_autoencoder(X_scaled, latent_dim, epochs=300, lr=0.001):
    X_tensor = torch.tensor(X_scaled, dtype=torch.float32)
```

```

model = Autoencoder(X_scaled.shape[1], latent_dim)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=lr)

for epoch in range(epochs):
    optimizer.zero_grad()
    output, _ = model(X_tensor)
    loss = criterion(output, X_tensor)
    loss.backward()
    optimizer.step()
    if (epoch + 1) % 50 == 0:
        print(f'Latent dim {latent_dim}, epoch {epoch+1}/{epochs}, loss: {loss.item():.4f}')

model.eval()
with torch.no_grad():
    _, latent = model(X_tensor)
return latent.numpy(), loss.item()

```

```

X_ae2, loss2 = train_autoencoder(X_scaled, 2)
plot_2d(X_ae2, 'Autoencoder (2 neurons)', 'autoencoder_2d.png')
print("Autoencoder final reconstruction loss (2 neurons):", loss2)

```

```

X_ae3, loss3 = train_autoencoder(X_scaled, 3)
plot_3d(X_ae3, 'Autoencoder (3 neurons)', 'autoencoder_3d.png')
print("Autoencoder final reconstruction loss (3 neurons):", loss3)

```

```

perplexities = [20, 30, 40, 50, 60]

```

```

for p in perplexities:
    tsne2 = TSNE(n_components=2, perplexity=p, init='pca', random_state=42)
    X_tsne2 = tsne2.fit_transform(X_scaled)
    plot_2d(X_tsne2, f't-SNE 2D (perplexity={p})', f'tsne_2d_perp {p}.png')

```

```

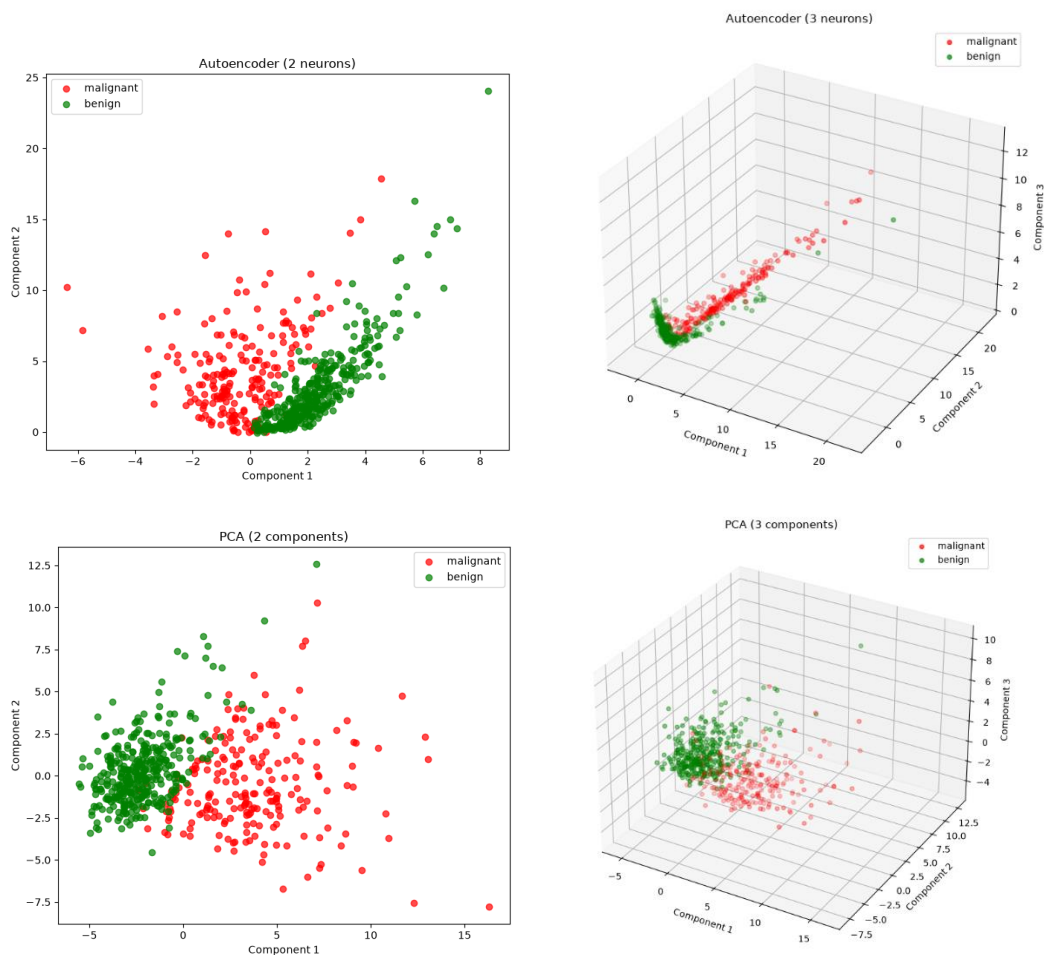
for p in perplexities:
    tsne3 = TSNE(n_components=3, perplexity=p, init='pca', random_state=42)

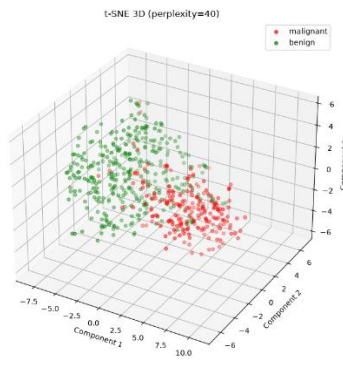
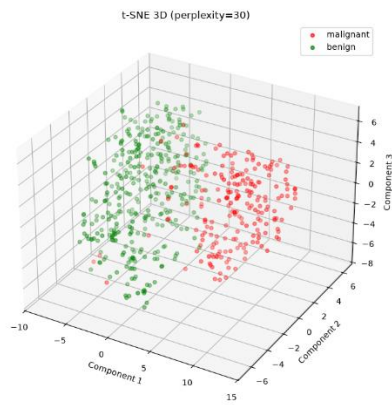
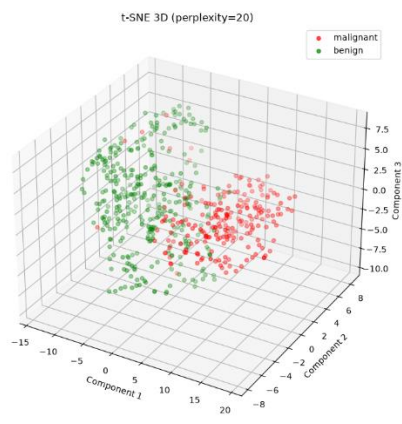
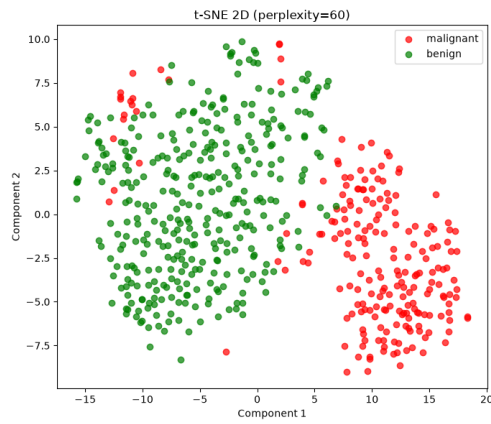
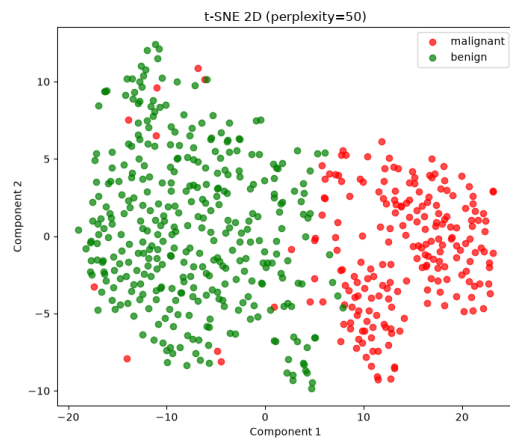
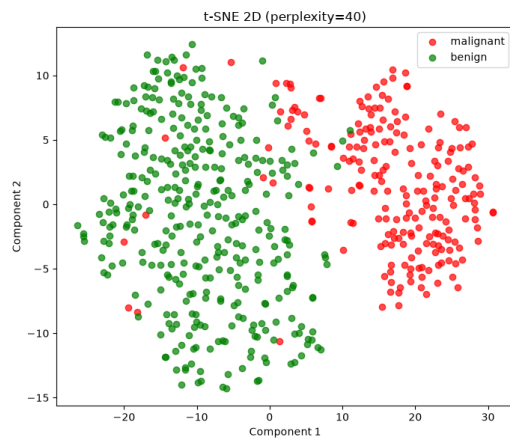
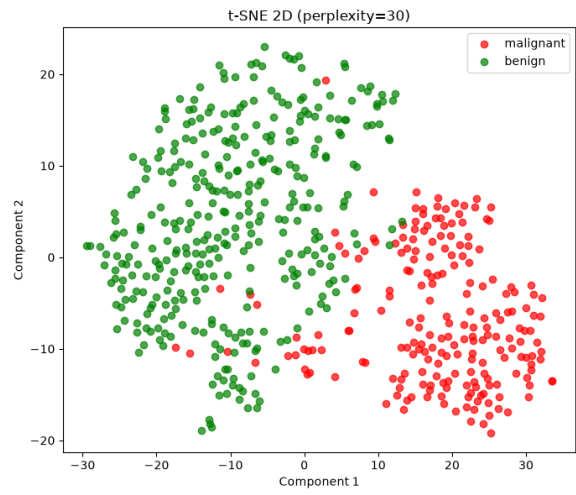
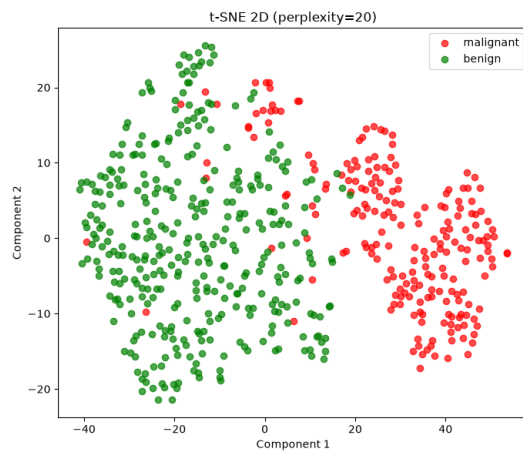
```

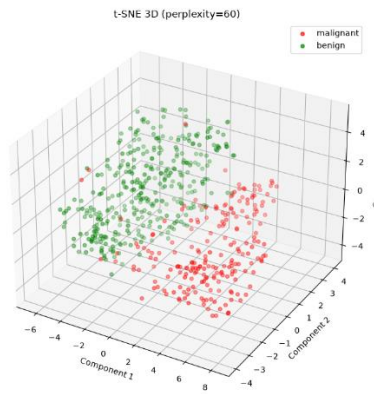
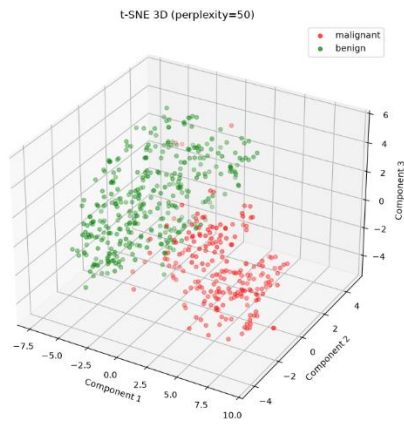
```
X_tsne3 = tsne3.fit_transform(X_scaled)
```

```
plot_3d(X_tsne3, f't-SNE 3D (perplexity={p})', f'tsne_3d_perp{p}.png')
```

```
PCA explained variance ratio (2 comp): 0.6324320765155946
PCA explained variance ratio (3 comp): 0.7263637090899084
Latent dim 2, epoch 50/300, loss: 0.8672
Latent dim 2, epoch 100/300, loss: 0.5682
Latent dim 2, epoch 150/300, loss: 0.4474
Latent dim 2, epoch 200/300, loss: 0.3916
Latent dim 2, epoch 250/300, loss: 0.3701
Latent dim 2, epoch 300/300, loss: 0.3566
Autoencoder final reconstruction loss (2 neurons): 0.35656672716140747
Latent dim 3, epoch 50/300, loss: 0.9744
Latent dim 3, epoch 100/300, loss: 0.5862
Latent dim 3, epoch 150/300, loss: 0.4053
Latent dim 3, epoch 200/300, loss: 0.3666
Latent dim 3, epoch 250/300, loss: 0.3455
Latent dim 3, epoch 300/300, loss: 0.3289
Autoencoder final reconstruction loss (3 neurons): 0.32893750071525574
```







к датасету WDBC (569 объектов, 30 признаков, 2 класса) были применены автоэнкодер, PCA и t-SNE.

- Автоэнкодер (30→16→8→2/3, Adam, MSE) дал финальную ошибку реконструкции 0.357 (2 нейрона) и 0.329 (3 нейрона).
- PCA (реализован вручную и через sklearn) объяснил 63.24% дисперсии при 2 компонентах и 72.64% при 3, то есть потери составили 36.76% и 27.36% соответственно. Несмотря на потери, классы различимы, особенно в 3D.
- Сравнение показало, что автоэнкодер не дал выигрыша перед PCA — его ошибка сопоставима или даже выше потерь PCA, что говорит о преимущественно линейной структуре данных WDBC.
- t-SNE протестирован при perplexity от 20 до 60; во всех случаях классы разделились чётче, чем у PCA и автоэнкодера. Оптимальными признаны значения 30–40 — наиболее компактное и стабильное разделение кластеров.

Итог: t-SNE даёт лучшее визуальное разделение классов, PCA — быстрая и интерпретируемая базовая визуализация, автоэнкодер — гибкий, но не давший преимущества метод для этого линейного по структуре датасета.

Вывод: научилась применять автоэнкодеры для осуществления визуализации данных и их анализа